

2-MySQL-Advanced

鸣谢：黑马程序员。（【黑马程序员 MySQL数据库入门到精通，从mysql安装到mysql高级、mysql优化全囊括】https://www.bilibili.com/video/BV1Kr4y1i7ru?vd_source=b7f14ba5e783353d06a99352d23ebca9）

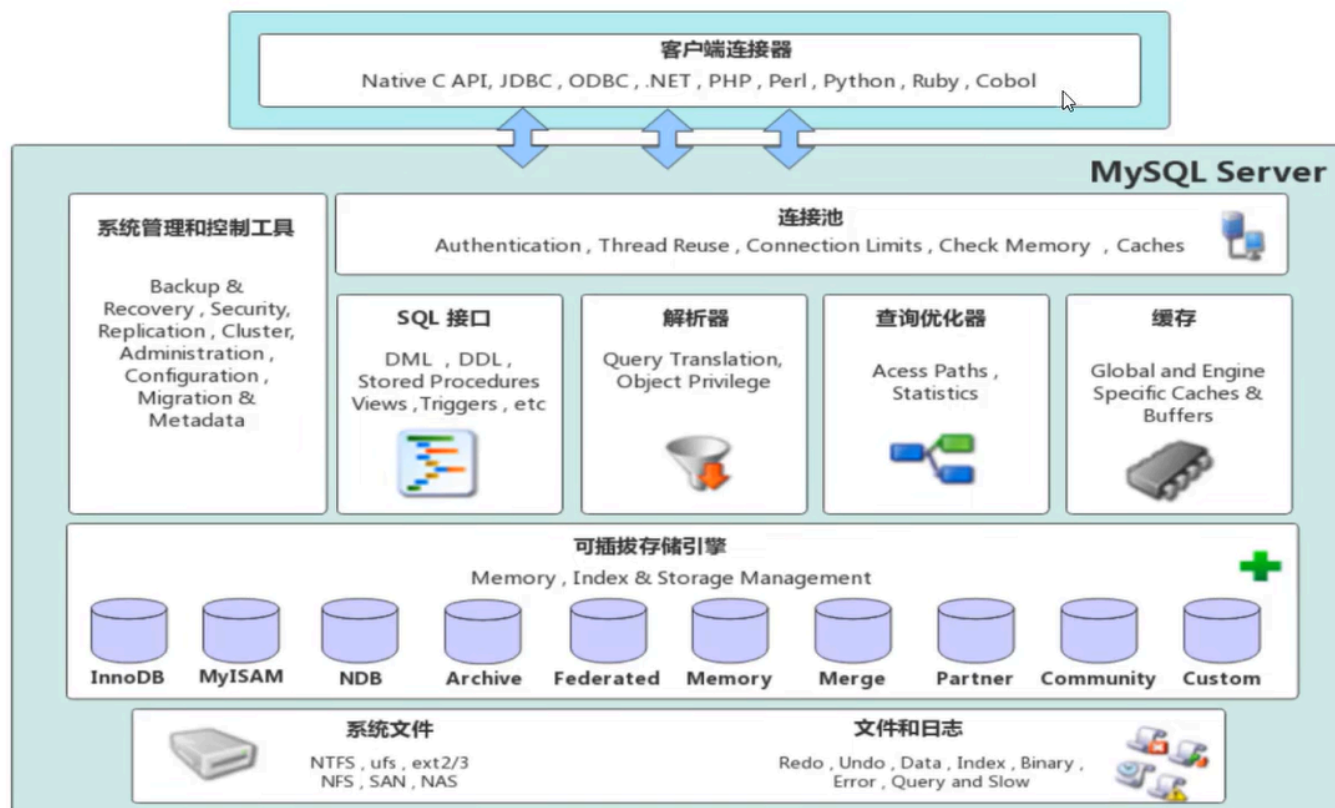


一、存储引擎

1.MySQL体系结构

自上而下分别是连接层、服务层、引擎层、存储层。

- **连接层**：包含一些客户端和链接服务，主要完成一些类似于连接处理、授权认证及相关的安全方案，服务器也会为安全接入的每个客户端验证它所具有的操作权限。
- **服务层**：主要完成大多数的核心功能（如SQL接口），并完成缓存查询、SQL分析和优化、部分内置函数的执行。所有跨存储引擎的功能也在这一层实现，比如过程、函数等。
- **引擎层**：存储引擎负责MySQL中数据的存储和提取，服务器通过API和存储引擎进行通信。不同的存储引擎具备不同功能，我们可以根据需求选择合适的存储引擎。
- **存储层**：主要是将数据存储于文件系统之上，并完成与存储引擎的交互。



2. 存储引擎简介

- 存储引擎就是存储数据、建立索引、更新和查询数据等技术的实现方式。
- 存储引擎是基于表的，而不是基于库的，因此存储引擎也被称为表类型。

2.1 在建表时指定存储引擎

若不指定存储引擎，则：

- MySQL 5.5起：默认是InnoDB
- MySQL 5.5之前：默认是MyISAM

```
1 create table 表名(  
2     字段1 字段1类型,  
3     ...  
4     字段n 字段n类型  
5 )engine=InnoDB;
```

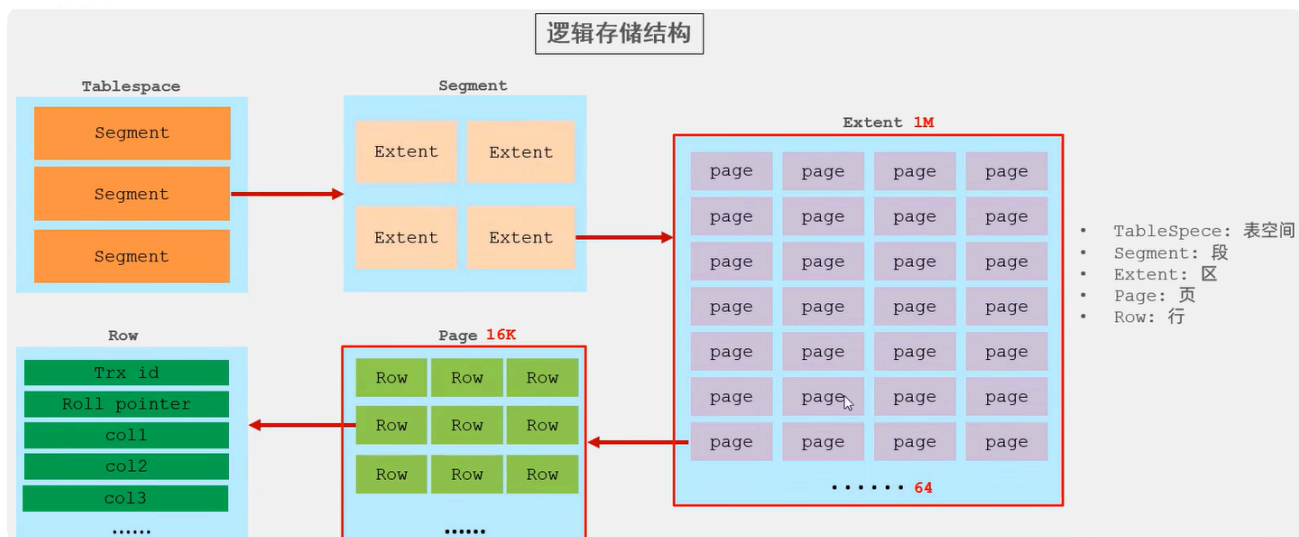
2.2 查看当前数据库支持的存储引擎

```
1 | show engines;
```

3.存储引擎特点

3.1 InnoDB

- **介绍：**InnoDB是一种兼顾高可靠性和高性能的通用存储引擎。
- **特点：**
 - DML操作支持ACID模型，支持**事务**。
 - 支持**行锁**，提高并发访问性能。
 - 支持**外键**约束，保证数据的完整性和正确性。
- **文件：**
 - `xxx.ibd`：xxx是表名，InnoDB引擎的每张表都会对应这样一个表空间文件，存储该表的表结构（frm、sdi）、数据和索引。
 - 参数：`innodb_file_per_table`。
- **逻辑存储结构：**



3.2 MyISAM

- **介绍：** MyISAM是MySQL早期的默认存储引擎。
- **特点：**
 - 不支持事务，不支持外键。
 - 支持表锁，不支持行锁。
 - 访问速度快。
- **文件：**
 - `xxx.sdi`：存储表结构信息。
 - `xxx.MYD`：存储数据。
 - `xxx.MYI`：存储索引。

3.3 Memory

- **介绍：** 表数据存储在内存中，由于受到硬件或断电问题的影响，只能将使用Memory引擎的表作为临时表或缓存使用。
- **特点：**
 - 存放在内存中，访问速度快。
 - 支持显式哈希索引。
- **文件：** 只有 `xxx.sdi` 这一文件，存储表结构信息。

3.4 总结

特点	InnoDB	MyISAM	Memory
存储限制	64TB	取决于操作系统和文件系统	取决于内存和参数
支持事务	✓	×	×
锁机制	行锁	表锁	表锁
支持外键	✓	×	×
B+tree索引	✓	✓	✓
显式Hash索引	×	×	✓
全文索引	✓ (MySQL 5.6之后)	✓	×
磁盘空间使用	高	低	不占用
内存使用	高	低	中
批量插入速度	低	高	高

4.存储引擎选择

4.1 原则

- 根据应用系统的特点来选择存储引擎。
- 对于复杂的应用系统，可以根据实际情况选择多种存储引擎进行组合。

4.2 具体选择

- InnoDB：应用对于事务的完整性要求较高，在并发条件下要求数据的一致性，数据操作除了插入和查询之外，还包含很多更新和删除操作。
- MyISAM：
 - 应用以读和插入操作为主，并且对事务的完整性和并发性要求不高。
 - 当今更好的选择是MongoDB。
- Memory：
 - 将所有数据保存在内存中，访问速度快，通常用于临时表或缓存。缺点是对表的大小有限制，太大的表无法缓存在内存中，而且无法保障数据的安全性。
 - 当今更好的选择是Redis。

二、索引

1.概述

- 除了数据，数据库系统还维护着满足特定查找算法的数据结构，这些数据结构以某种方式引用/指向数据，这样数据库系统就可以基于这些数据结构来实现高级查找算法。
- 索引(index)就是这样一种有助于MySQL高效获取数据的有序数据结构。
- 演示：



备注：上述二叉树索引结构的只是一个示意图，并不是真实的索引结构。

高级软件人才培养

- 优缺点：

优点	缺点
提高数据检索的效率，降低数据库的磁盘IO成本	索引列需要占用一部分空间
通过索引列对数据进行排序，降低数据排序的成本，降低CPU消耗	索引大大提高了查询效率，却也降低了更新表的速度，对表进行增删改操作时效率降低

2.索引结构

2.1 介绍

MySQL的索引是在存储引擎层实现的，不同的存储引擎有着不同的结构，主要包含以下几种：

- B+树索引（默认）：最常见的索引类型，大部分引擎都支持B+树索引。
- Hash索引：底层数据结构是用哈希表实现的，只有精确匹配索引列的查询才有效，不支持范围查询。
- R-tree（空间索引）：空间索引是MyISAM引擎的一个特殊索引类型，主要用于地理空间数据类型，一般情况下较少使用。
- Full-text（全文索引）：是一种通过建立倒排索引，快速匹配文档的方式，类似于Apache Lucene、Solr、ES。

	InnoDB	MyISAM	Memory
B+树索引	✓	✓	✓
Hash索引	×	×	✓
空间索引	×	✓	×
全文索引	✓ (MySQL 5.6之后)	✓	×

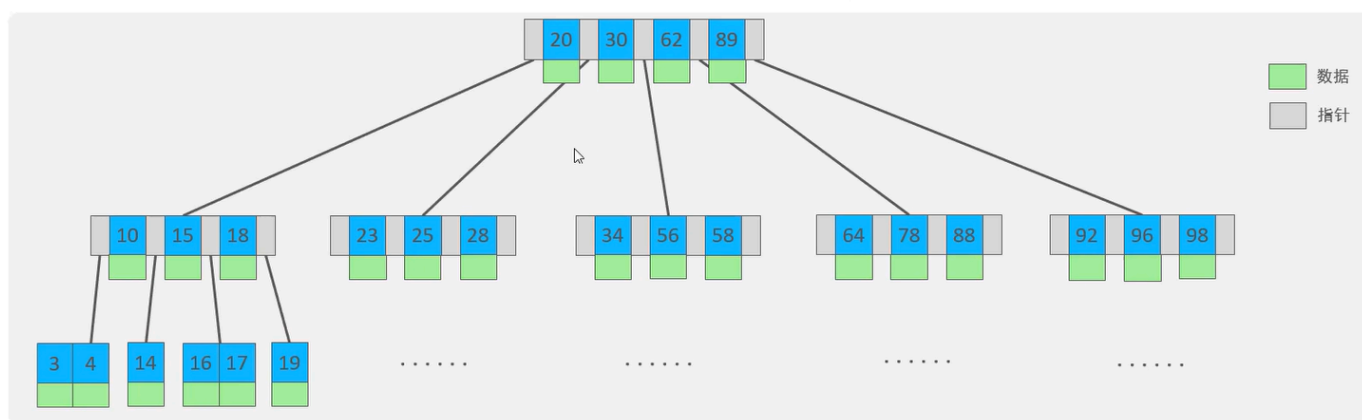
2.2 B树

B树是一种自平衡的多路搜索树，它通过控制每个结点的子结点数量，来保证树的高度较低，从而实现高效的插入、删除和查找操作。

一棵 m 阶B树($m \geq 2$)必须满足以下性质：

1. 如果根结点不是叶子结点，则至少有2个子结点。
2. 每个非根非叶结点至少有 $\lceil m/2 \rceil$ 个子结点，最多有 m 个子结点。
3. 所有叶子结点都在同一层，并且不包含任何子结点。
4. 每个结点包含 k 个关键字和 $k+1$ 个指向子结点的指针，其中 $\lceil m/2 \rceil - 1 \leq k \leq m-1$ 。关键字按升序排列，子树中的所有关键字都分别小于、等于或大于结点内的关键字。

以一颗最大度数（max-degree）为5(5阶)的b-tree为例(每个节点最多存储4个key, 5个指针)：



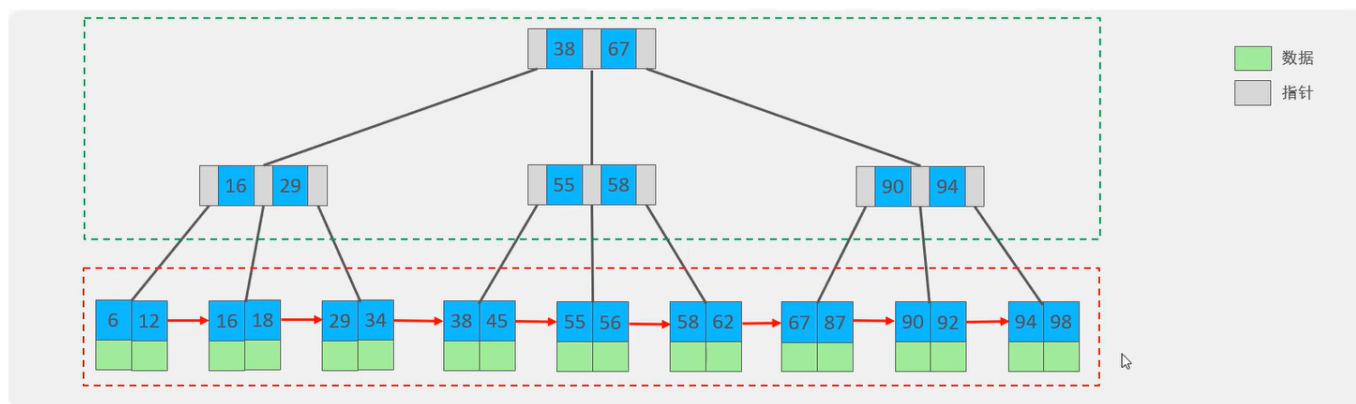
2.3 B+ 树

B+树是B树的一种变形形式，B+树上的叶子结点存储关键字以及指向相应记录的指针，叶子结点以上的各层仅作为索引使用。

一棵 m 阶B+树定义如下：

1. 每个结点至多有 m 个子结点。
2. 除根结点外，每个结点至少有 $\lceil m/2 \rceil$ 个子结点，根结点至少有2个子结点。
3. 有 k 个子结点的结点必有 k 个关键字。

以一颗最大度数 (max-degree) 为4 (4阶) 的b+tree为例:

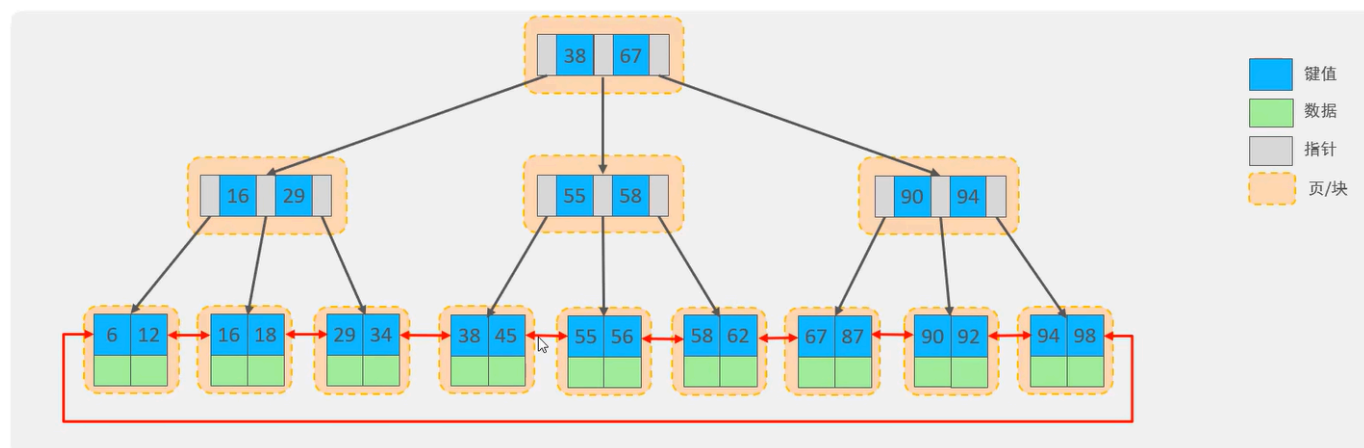


B+ 树相对于B树的区别:

- 所有数据都会出现在叶子结点。
- 所有叶子结点形成一个单向链表。

2.4 MySQL中的B+ 树

MySQL索引数据结构对经典B+ 树进行了优化, 使得所有叶子结点形成一个双向循环链表, 提高区间访问性能。

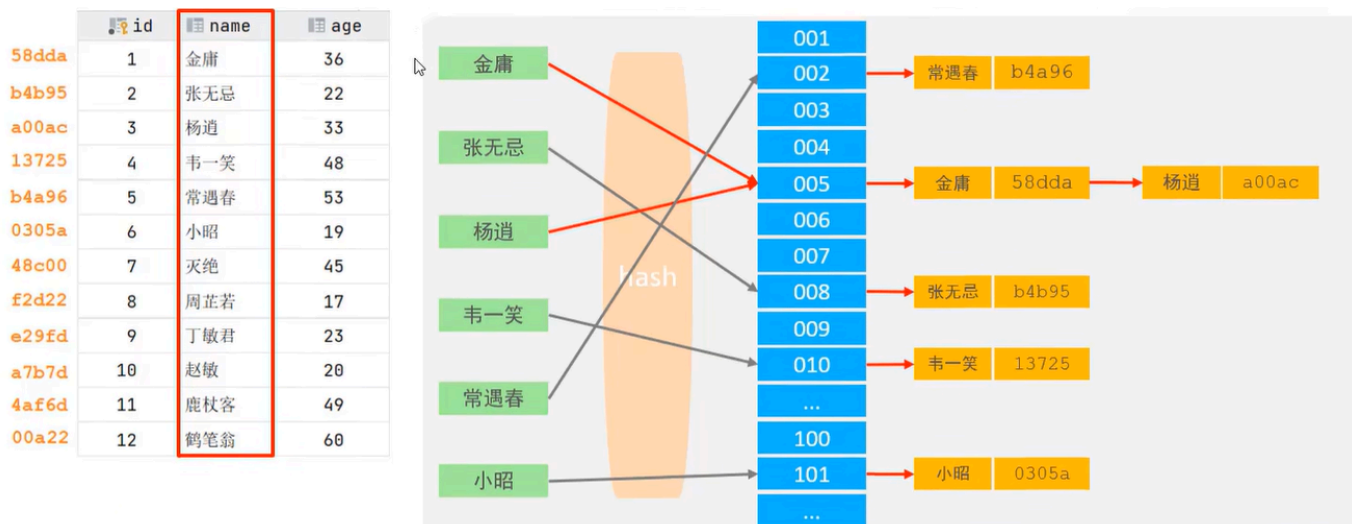


2.5 Hash索引

P1 概述

Hash索引就是采用一定的Hash算法，将关键字换算成新的Hash值并映射到对应槽位上，然后存储在Hash表中。

如果多个关键字映射到同一槽位上，就产生了Hash冲突（也称Hash碰撞），这时候可以使用链表来解决。



P2 特点

- Hash索引只能用于对等比较（=, in），不支持范围查询（between, >, <, ...）。
- 无法利用Hash索引完成排序操作。
- 查询效率高，通常只需要进行一次检索即可，效率通常高于B+树索引。

P3 支持Hash索引的引擎

Memory引擎支持Hash索引，而InnoDB引擎具有自适应Hash功能，可以根据B+树索引在指定条件下自动构建Hash索引。

3.索引分类

4.索引语法

5.SQL性能分析

6.索引使用

7.索引设计原则

三、SQL优化

四、视图/存储过程/触发器

五、锁

六、InnoDB核心

七、MySQL管理